X Monitor — Interview Notes

A real-time market news tracker: what you built, how to explain it, and the questions you'll get

X Monitor: Real-Time Market Sentiment & Tweet Tracker | Next.js, AWS S3, Pusher, Chrome MV3

- Built a webhook ingestion pipeline that normalizes and de-duplicates **1,000+ tweets a day** from **20+ channels** into an S3 store with optimistic-concurrency writes, pushing breaking market news to a **Chrome Extension (MV3)** in **under 500 ms**.
- Architected event-driven streaming (Pusher WebSockets, HTTP 304 ETag caching), cutting daily outbound bandwidth by **99%+ (10 GB+ to ~1 MB)** and eliminating **6,500+ redundant serverless invocations**.

How to read these notes

Samjho — explanation in Hinglish, for you Interview mein bolo — simple English lines to say

Example — a small case to mention Careful — places interviewers will push

Contents

1. The pitch: 30-second and 2-minute versions
2. The whole system on one page (architecture diagram)
3. What an ingestion pipeline is
4. Normalize and de-duplicate: how it works
5. "1,000+ tweets a day from 20+ channels"
6. Why S3?
7. Optimistic-concurrency writes
8. "Under 500 ms": how it's calculated
9. Event-driven streaming: Pusher + ETag/304
10. The numbers: 10 GB → 1 MB and 6,500+ invocations
11. Chrome Extension MV3 details
12. How to draw the architecture on a whiteboard
13. Interview questions and answers (30)
14. Weak spots, trade-offs and what you'd improve
15. One-page cheat sheet

1. The pitch

SAMJHO

Simple si baat hai: stock market ke time pe Indian financial news accounts (Moneycontrol, NDTV Profit, analysts) X pe jo bhi breaking post karte hain, woh turant trader ke browser mein ek floating panel aur desktop notification ke roop mein aa jaata hai. Tweets ek third-party service (TwitterAPI.io) se webhook ke through server pe aate hain; server unhe saaf karta hai (normalize), duplicates hata deta hai, S3 mein save karta hai, aur Pusher se seedha Chrome extension ko push kar deta hai. Pehle extension har 2 second mein server se poochta tha "kuch naya hai?" — usme bahut bandwidth aur serverless calls waste hoti thi. Push model se woh waste almost zero ho gaya.

30-second version

INTERVIEW MEIN BOLO

"X Monitor is a real-time news tracker for Indian stock-market traders. During market hours, posts from financial news accounts on X arrive at my Next.js backend through a webhook. The backend cleans them into one standard format, removes duplicates, saves them in S3, and pushes them to a Chrome extension over Pusher WebSockets. A new post shows up as a desktop notification in under half a second. The first version polled the server every two seconds. Moving to push plus ETag caching cut bandwidth by over 99%."

2-minute version (problem → design → result)

INTERVIEW MEIN BOLO

Problem. "In Indian markets, a headline like 'RBI cuts repo rate' moves prices within seconds. Traders don't want to keep refreshing X. They want the news pushed to them the moment it's posted, from a fixed list of trusted accounts."

Ingestion. "I use TwitterAPI.io's user stream. It calls my webhook, `/api/twitter-webhook` on Vercel, every time a monitored account posts. The webhook checks a secret, then normalizes the payload, because the provider sends several different JSON shapes. It de-duplicates by tweet ID and merges the result into one JSON document in S3."

Concurrency. "S3 has no append. If two webhook calls land at the same time, one could overwrite the other. So I use optimistic concurrency: read the object with its ETag, then write with `If-Match`. If someone else wrote in between, S3 returns 412, and I re-read and retry."

Delivery. "After saving, the webhook publishes the new tweets to a Pusher channel. The extension's service worker holds a WebSocket to Pusher, so the tweet lands in the browser in a few hundred milliseconds, and it raises a desktop notification."

Efficiency. "The first version polled every two seconds and downloaded the whole feed each time. I added ETag/304 responses, so an unchanged poll costs zero bytes of body. Then I moved to push, so the extension makes one catch-up request when it connects instead of thousands of polls. Daily outbound data went from over 10 GB to about 1 MB, and thousands of serverless calls a day disappeared."

2. The whole system on one page

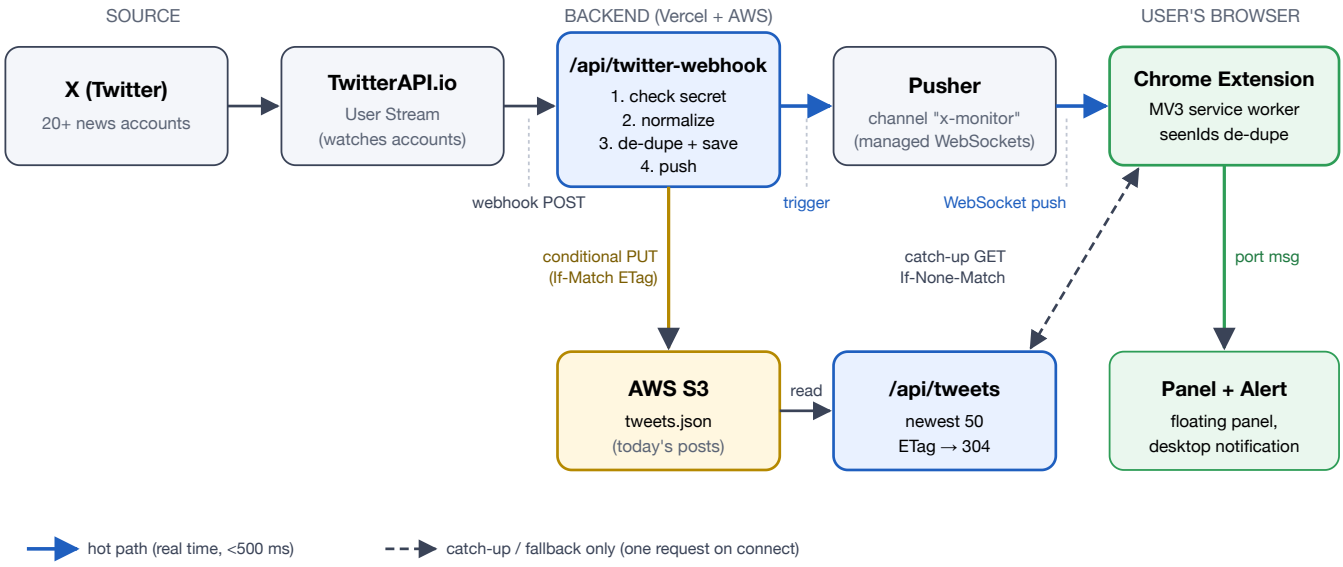


Figure 1. End-to-end flow. Blue arrows = the real-time hot path. Dashed = the catch-up read.

The flow in 6 steps

1. **Source.** A monitored account posts on X. TwitterAPI.io's User Stream sees it.
2. **Webhook.** TwitterAPI.io sends an HTTP POST to `/api/twitter-webhook?key=SECRET`. Vercel runs the function.
3. **Clean.** The secret is checked with `timingSafeEqual`, then `normalizePayload()` turns the payload into a standard tweet object.
4. **Save.** `addTweets()` merges the new tweets into `tweets.json` in S3: drop duplicates by ID, drop yesterday's posts, newest first, cap at 1,000. It writes with a conditional PUT.
5. **Push.** `pusher.trigger('x-monitor', 'new-tweets', …)`, at most 2 tweets per event to stay under Pusher's 10 KB message limit.
6. **Show.** The extension's service worker gets the WebSocket message, skips IDs it has already seen, updates the panel and badge, and fires a desktop notification.

> **SAMJHO**
> Ek line mein: **"Data aata hai webhook se, rehta hai S3 mein, jaata hai Pusher se."** S3 sirf "memory" hai (record rakhne ke liye, aur jab extension naya khule toh purane tweets dikhane ke liye). Real-time delivery Pusher karta hai. `/api/tweets` sirf backup hai: extension connect hote hi ek baar "catch-up" call karta hai, ya Pusher band ho toh fallback polling.

Supporting pieces (mention if asked)

- **Market-hours gate**: storing and polling run only 9:00–15:30 IST, with 5 minutes of grace to 15:35. Old posts are hidden at 08:55 every morning. That boundary is *computed* (`retentionCutoff()`), not a cron job, so server and extension agree to the second.
- **Manual ON mode**: a 5-minute override for after hours, which then falls back to Auto by itself.
- **WebSocket bridge** (`scripts/stream-listener.mjs`): another way in. It listens to TwitterAPI.io's WebSocket and forwards to the same webhook, so both sources share one pipeline.

- **Cloudflare edge shield** (optional): a Worker in front of the webhook answers off-hours deliveries at the edge, so Vercel is never invoked for them.

3. What is an ingestion pipeline?

SAMJHO

"Ingest" matlab **andar lena**. Ingestion pipeline matlab bahar se aane wale raw data ko andar lena, saaf karna, aur kahin store karna, step-by-step, ek assembly line ki tarah.

Analogy: Ek post office socho. Har courier company ka parcel alag shape aur label mein aata hai (raw data). Counter pe staff check karta hai ki parcel genuine hai (auth), sab pe ek standard label lagata hai (normalize), same parcel do baar aaya toh ek hata deta hai (de-dupe), shelf pe rakhta hai (store), aur customer ko SMS bhejta hai "aapka parcel aa gaya" (push notification).

"Webhook" ingestion means the provider *calls you* when there's new data. You don't keep asking the provider. That's "push", the opposite of "pull" or "poll".

Stage	What happens in X Monitor	Code
1. Receive	TwitterAPI.io sends a POST with one or more tweets	`POST /api/twitter-webhook`
2. Authenticate	Secret in the URL (`?key=`) or `x-api-key` header, compared with `timingSafeEqual` . With no secret configured, every write is refused.	`secretMatches()`
3. Validate	Empty or non-JSON body = provider's validation ping → reply 200 and do nothing	route.js
4. Normalize	Any payload shape → one standard tweet object	`normalizePayload()`
5. De-dupe + store	Merge into S3 by tweet ID, conditional write	`addTweets()` → `s3Update()`
6. Fan-out	Publish to the Pusher channel	`pusher.trigger()`
7. Acknowledge	200 on success. 500 on a store failure, so the provider retries instead of dropping the batch.	route.js

INTERVIEW MEIN BOLO

"Ingestion pipeline just means the path data takes from the outside world into my system. Mine is webhook-based: the tweet provider calls my endpoint, and I authenticate, validate, normalize, de-duplicate, store and broadcast, in that order. Each step is small and does one job, so it's easy to test and reason about."

EXAMPLE

At 10:42:05 AM, @NDTVProfit posts "Sensex jumps 600 points after RBI policy". At about 10:42:06 TwitterAPI.io POSTs it to my webhook. Within a few hundred milliseconds it's saved in S3 and pushed, and the trader sees a desktop notification: "NDTV Profit: Sensex jumps 600 points…"

4. Normalize and de-duplicate

4a. Normalize: many shapes in, one shape out

SAMJHO

TwitterAPI.io har baar same format mein data nahi bhejta. Kabhi tweets `payload.tweets` mein aate hain, kabhi `payload.data` mein, kabhi ek single object. Username kabhi `user.screen_name` mein hota hai, kabhi `author.userName` mein. Normalize karna matlab: **sabko ek hi standard shape mein convert karna**, taaki extension ko sirf ek format samajhna pade. Jaise alag-alag languages ke documents ko ek common language mein translate karna.

Raw payload A (one shape)

```
{ "tweets": [{
    "id": "1849…",
    "text": "RBI keeps repo rate at 6.5%",
    "createdAt": "2026-10-21T04:30:12Z",
    "author": { "userName": "NDTVProfit",
                "name": "NDTV Profit" }
}]}
```

Raw payload B (another shape)

```
{ "data": {
    "tweetId": "1849…",
    "full_text": "RBI keeps repo rate at
6.5%",
    "user_id": "420943164",
    "favorite_count": 12
}}
```

After `normalizeTweet()`, always this shape:

```
{
  "id": "1849…",
  "text": "RBI keeps repo rate at 6.5%",
  "created_at": "2026-10-21T04:30:12Z",
  "author": {
    "username": "NDTVProfit",
    "name": "NDTV Profit",
    "profile_image_url": "…"
  },
  "media": [],
  "metrics": { "likes": 12,
               "retweets": 0, "replies": 0
},
  "received_at": "2026-10-21T04:30:13.120Z"
}
```

What normalization does in your code:

- `extractRawTweets()` unwraps the envelope (array, `.tweets`, `.data`, `.tweet`, or a single tweet).
- `resolveAuthor()` finds the author. The **numeric user ID** is the strongest signal, because handles can change and IDs don't. It falls back to the handle, and then to fuzzy matching on the *display name*.
- `extractMedia()` collects photos and videos from four different places a payload can keep them, de-duplicated by URL.
- Adds `received_at`, the server time the tweet arrived. Useful for measuring latency.

EXAMPLE: A REAL BUG YOU FIXED (GOOD STORY)

The first version of the fuzzy matcher searched the tweet **body** as well as the author's name. Any tweet that merely mentioned "ANI" or "Moneycontrol" got credited to that account. Example: a Yatin Mota tweet saying "as reported by Moneycontrol…" showed up as Moneycontrol. The fix: match on the display name only, and prefer the stable numeric user ID.

4b. De-duplicate: the same tweet must appear only once

SAMJHO

Duplicate kyun aate hain? (1) Webhook providers **retry** karte hain: agar humara server slow tha ya 500 diya, toh same tweet dobara bhejenge. (2) Do ingestion sources hain (webhook + WebSocket bridge), dono same tweet la sakte hain. (3) Extension ko same tweet Pusher se bhi milta hai aur catch-up GET se bhi. Har tweet ka ek unique `id` hota hai, toh hum ek **Set of IDs** rakhte hain: agar ID pehle se hai, skip.

De-dupe happens at two layers (defense in depth):

```
// Server: src/lib/store.mjs, merge()
const ids = new Set(kept.map(t => t.id));
const unique = incoming.filter(t => t.id && !ids.has(t.id) && !isExpired(t));
const list = [...unique, ...kept].slice(0, MAX_TWEETS);   // newest first, cap 1000

// Extension: background.js
const newTweets = incoming.filter(t => !seenIds.has(t.id));   // notify only once
```

INTERVIEW MEIN BOLO

"Webhooks are at-least-once: the provider may send the same tweet twice when it retries. So I made the write **idempotent**. The merge uses a Set of tweet IDs, and sending the same tweet twice leaves the store unchanged. If nothing changed I skip the S3 write entirely. The extension keeps its own `seenIds` set too, so a tweet that arrives over both Pusher and the catch-up request triggers only one notification."

EXAMPLE

The webhook saves tweet `1849…` but takes too long to reply, so TwitterAPI.io retries. The second delivery reaches `merge()`, finds the ID already in the Set, and gets `added = 0, changed = false`. No S3 write happens, and the user still sees one notification.

5. "1,000+ tweets a day from 20+ channels"

SAMJHO

"Channel" yahan matlab **ek monitored X account** (news handle ya analyst), jaise @moneycontrolcom, @NDTVProfit, @LiveLawIndia, CNBC-TV18, ET NOW, Zee Business, Livemint, PTI, ANI, Bloomberg, aur analysts jaise Yatin Mota. News accounts market ke time bahut post karte hain: ek bada handle 6.5 ghante mein 100+ posts kar sakta hai. Toh 20 accounts × ~50 average = 1,000+ per day. Store ka cap bhi isi hisaab se `MAX_TWEETS = 1000` rakha hai ("comfortably a full market day").

INTERVIEW MEIN BOLO

"By channels I mean the X accounts I monitor: Indian financial media like Moneycontrol, NDTV Profit and CNBC-TV18, plus a few market analysts. News handles post very often during market hours, so across 20-odd accounts that's over a thousand posts a day. I sized the store for that: it keeps up to 1,000 of today's posts and clears everything at 8:55 AM the next morning."

CAREFUL

Know your own numbers before the interview. The extension has display labels for about 19–20 accounts, but which accounts actually stream is set on the TwitterAPI.io plan. The plan is billed per account, and the README notes six accounts streaming as of Sep 2026. If you're asked "which 20?" or "is that live today?", answer honestly. For example: "The system is built for 20+ and was run with that many. The paid plan currently streams N accounts, and changing the set is a dashboard setting, not a code change." A calm, truthful answer beats a number you can't back up.

6. Why S3?

SAMJHO

Vercel serverless functions ki **apni koi permanent disk nahi hoti**. Har request ek naye ya alag container mein chal sakti hai, aur local file request khatam hote hi gayab ho sakti hai. Toh data ko kahin bahar rakhna padega. Humara data chhota hai: ek din ke ~1,000 tweets, ek JSON file, kuch sau KB. Iske liye poora database (Postgres/Mongo) chalana overkill hai. S3 sasta hai, kabhi down nahi hota (99.99% availability), koi server manage nahi karna padta, aur sabse important: **S3 mein ETag aur conditional requests built-in hain**, jinse caching (304) aur safe writes (If-Match) dono free mein mil jaate hain.

Option	Why not / why
Local file on Vercel	No. Serverless filesystem is temporary, and each function instance has its own copy.
Postgres / MongoDB	Works, but it's a server to run, a connection pool to manage from serverless, and extra cost, all for one small list.
Redis	Fast, but another paid service, and you'd still want something durable behind it.
DynamoDB	A good choice if this grows (one row per tweet, conditional writes too). More setup for v1.
S3 (chosen)	Cheap, durable (11 nines), serverless, strong read-after-write consistency, **native ETag + If-Match / If-None-Match**. One object = the whole day's feed.

INTERVIEW MEIN BOLO

"My backend is serverless on Vercel, so there's no local disk that survives between requests. I needed an external store. The data is small, a single day's feed of around a thousand tweets, so I keep it as one JSON object in S3. S3 is cheap and durable and needs no server. It also supports ETags and conditional requests, which gave me two things for free: 304 caching on reads and safe concurrent writes with If-Match. If the product grew, say per-user feeds or months of history, I'd move to DynamoDB with one item per tweet."

EXTRA DETAIL THAT IMPRESSES

Reads sit behind a **1-second in-memory cache** and use a **conditional GET** (`IfNoneMatch: lastEtag`). If several requests hit the same warm function within a second, S3 is called once. And when S3 hasn't changed, it answers 304 with no body, so even the S3 call is cheap.

7. Optimistic-concurrency writes

SAMJHO

Problem: "lost update". S3 mein "append" nahi hota. File update karne ke liye poori file padho → usme naya tweet jodo → poori file wapas likho. Ab socho do webhooks *ek saath* aaye: dono ne same purani file padhi, dono ne apna-apna tweet joda, dono ne likha. Jo baad mein likhega, woh pehle wale ka tweet **mita dega**. Tweet gaya!

Solution: optimistic concurrency. "Optimistic" matlab hum maan ke chalte hain ki clash kam hi hoga, isliye pehle se lock nahi lagate. Bas likhte waqt check karte hain: "file abhi bhi wahi version hai na jo maine padha tha?" S3 har version ko ek **ETag** (fingerprint) deta hai. Hum likhte hain `If-Match: <etag>`. Agar beech mein kisi aur ne likh diya, toh ETag badal gaya hoga, aur S3 bolega **412 Precondition Failed**. Tab hum file dobara padhte hain (ab usme dusre ka tweet bhi hai), apna tweet jodte hain, aur phir try karte hain.

Analogy: Google Docs / Wikipedia edit conflict: "Is page ko aapke edit shuru karne ke baad kisi ne badal diya hai, please refresh karke dobara karo."

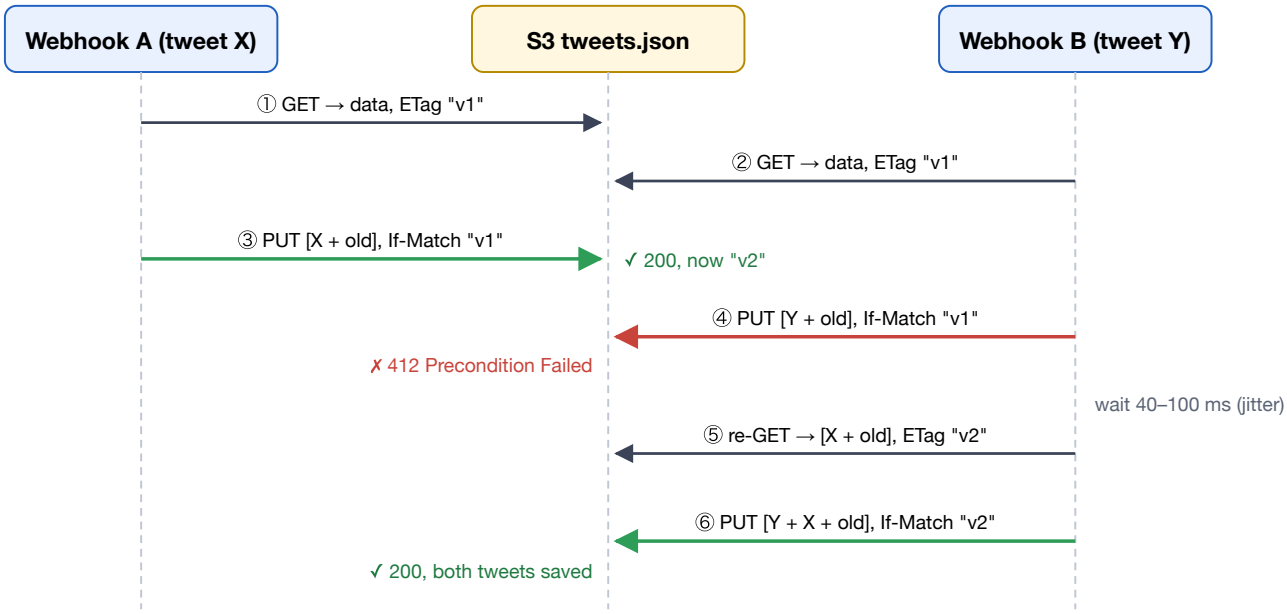


Figure 2. Two webhooks race. Without If-Match, step ④ would silently delete tweet X.

```
// src/lib/store.mjs, s3Update() (simplified)
for (let attempt = 1; attempt <= 6; attempt++) {
  const { tweets: current, etag } = await s3Fetch();          // read + version
  const { list, changed } = transform(current);               // merge / de-dupe
  if (!changed) return current;                               // nothing new → skip write
  try {
    await s3.send(new PutObjectCommand({ ..., Body: JSON.stringify(list),
      ...(etag ? { IfMatch: etag } : { IfNoneMatch: '*' }) })); // write only if unchanged
    return list;
  } catch (err) {
    if (!isConflict(err)) throw err;                          // real error → 500
    await sleep(40 * attempt + Math.random() * 60);           // lost the race → retry
```

```
  }
}
```

	Optimistic (what you used)	Pessimistic
Idea	No lock. Check the version at write time, retry on conflict.	Take a lock first, then no one else can write.
Good when	Conflicts are rare (a few webhooks a minute)	Conflicts are frequent and retries are expensive
Cost	Occasional retry	Lock service, deadlocks, lock timeouts
S3 support	Yes: `If-Match` / `If-None-Match: *` on PUT	No native locks

INTERVIEW MEIN BOLO

"S3 objects can't be appended to, so every write is read-modify-write. If two webhook calls overlap, the later write would silently erase the earlier one. That's the classic lost-update problem. I solved it with optimistic concurrency. I read the object along with its ETag, which is its version fingerprint, and write with `If-Match` set to that ETag. If anyone wrote in between, S3 rejects my write with 412. Then I re-read, merge again, and retry, up to six times, with a small random delay so the two writers don't collide again. I chose optimistic over locking because conflicts are rare here, and S3 has no locks anyway. For the very first write I use `If-None-Match: *`, which means 'create only if it doesn't exist'."

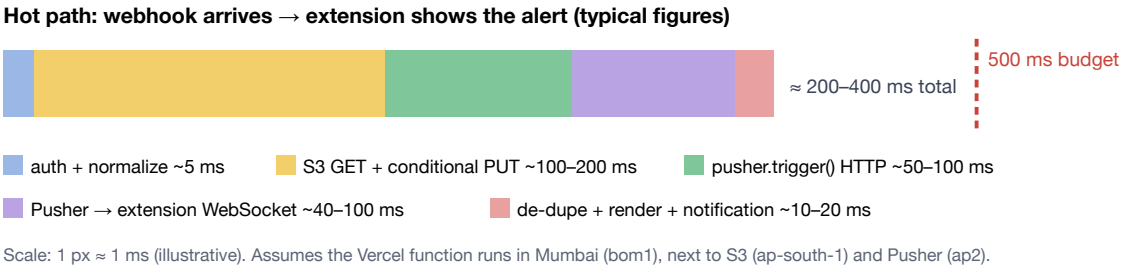
WHY RANDOM DELAY ("JITTER")?

If both losers waited exactly 50 ms, they'd retry at the same instant and clash again. A random 40–100 ms wait spreads them out, the same way two people in a corridor stop dancing left-right once one of them pauses.

8. "Under 500 ms": how it's calculated

SAMJHO

500 ms ka matlab: **jab tweet humare webhook pe pahuncha, tab se lekar jab tak extension ko mil gaya**, total time aadhe second se kam. Is time mein X → TwitterAPI.io wala hissa shaamil **nahi** hai, kyunki woh humare control mein nahi hai. Yeh clearly bolna, interviewer ko achha lagega ki aap apna measurement boundary jaante ho. Har step ka time roughly:



Scale: 1 px ≈ 1 ms (illustrative). Assumes the Vercel function runs in Mumbai (bom1), next to S3 (ap-south-1) and Pusher (ap2).

Figure 3. Latency budget. The S3 write is the biggest piece.

How to measure it (and how to describe it)

1. **Timestamps at both ends.** The server stamps every tweet with `received_at` in `normalizeTweet()`. When the extension gets the Pusher message, compute `Date.now() - Date.parse(tweet.received_at)` and log it. Both machines sync their clocks over NTP, so the skew is a few ms.
2. **Logs.** Compare the Vercel function log time ("Stored N tweets…", "Pushed N tweets…") with the extension console line "⚡ Instant delivery…".
3. **Pusher Debug Console** shows when each event was published, and Vercel shows function duration. Add the two.
4. Take **many samples** (for example a full market day) and quote the **p95**, not one lucky run. "p95 under 500 ms" means 95% of tweets arrived faster than that.

INTERVIEW MEIN BOLO

"I measured from the moment the webhook hits my server to the moment the extension receives the tweet. The server stamps each tweet with a `received_at` time, and the extension subtracts that from its own clock when the Pusher message arrives. Most of the time is the S3 write, roughly 100 to 200 ms. Then the Pusher publish, and the WebSocket hop to the browser. The function, the S3 bucket and the Pusher cluster are in the same region, so end to end it's typically a few hundred milliseconds and stays under 500 ms. The X-to-TwitterAPI.io hop is outside my control, so I don't count it."

CAREFUL: LIKELY FOLLOW-UP

"How would you make it faster?" Right now the push happens *after* the S3 write. Since S3 is the slowest step, you could publish to Pusher **first, or in parallel** (`Promise.all`) and persist afterwards. That cuts ~100–200 ms. The trade-off: a user might see a tweet that then fails to save. It's still in their extension, and the provider retries. Also, if you haven't actually logged these numbers yet, add the one-line latency log before the interview, so "how did you measure?" has a real answer.

CHECK BEFORE THE INTERVIEW: FUNCTION REGION

Your `vercel.json` doesn't set a region, and Vercel runs functions in **US East (iad1)** by default. If that's where yours run, every S3 call to Mumbai is a ~200 ms round trip, and GET + PUT alone could take ~400 ms. Open Vercel → Project → Settings → Functions → Region. If it isn't Mumbai (`bom1`), set it, or add `"regions": ["bom1"]` to `vercel.json` , and measure again. "I moved the function next to the data and latency dropped" is also a great thing to say in the interview.

9. Event-driven streaming: Pusher WebSockets + ETag/304

SAMJHO

Polling = car mein baitha bachcha har 2 second poochta hai "pahunch gaye? pahunch gaye?". 99% baar jawab "nahi" hota hai, phir bhi har baar poochne mein energy lagti hai. Humare case mein har "poochna" = ek Vercel function call + poora feed download.

Event-driven / push = papa bolte hain "jab pahunchenge, main bata dunga". Kuch hua (event) tabhi message aata hai, warna khamoshi. Isko ek **WebSocket** se karte hain: browser aur server ke beech ek open phone line jo khuli rehti hai, aur server jab chahe usme message bhej sakta hai.

Pusher kyun? Vercel serverless functions zyada der zinda nahi rehte, toh hazaron long-lived WebSocket connections hold nahi kar sakte. Pusher ek managed service hai jo yeh connections sambhalta hai. Humara server bas ek HTTP call karta hai (`trigger`), aur Pusher sab connected browsers tak pahuncha deta hai (fan-out).

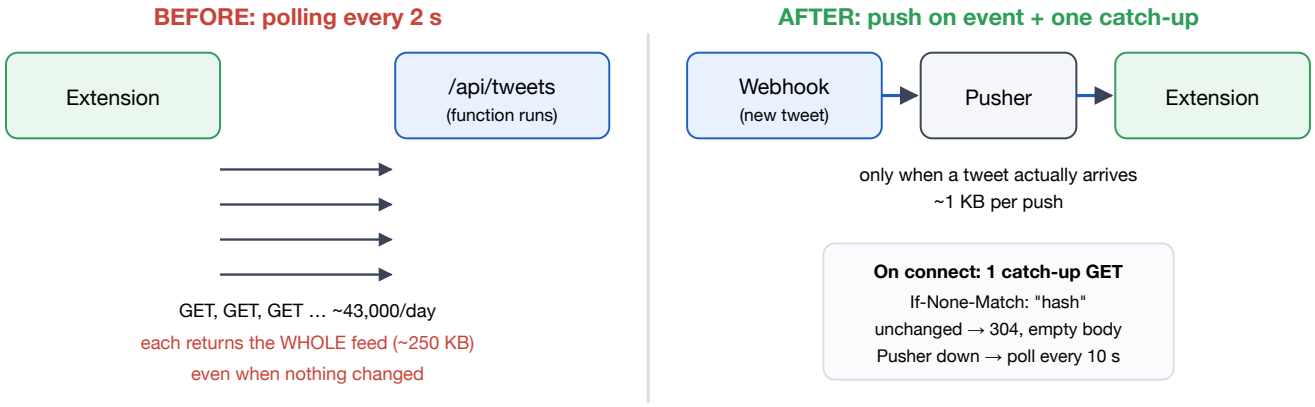


Figure 4. Pull vs push.

ETag / 304: "only send it if it changed"

SAMJHO

ETag = data ka **fingerprint**. Server pehli baar poora feed bhejta hai, saath mein bolta hai "iska fingerprint `"123"` hai". Agli baar browser poochta hai "mere paas `"123"` wala hai, kuch badla?". Agar nahi badla, server bas bolta hai **304 Not Modified**, bina body ke. Jaise teacher se poochna "sir, notes update hue?" — "nahi" bolna poore notes dobara photocopy karne se sasta hai.

```
→ GET /api/tweets?force=1                          ← 200 OK   ETag: "2891734412"
                                                      body: { tweets: [...50], feedHash:
"2891734412" }   (~30 KB)

→ GET /api/tweets?force=1                          ← 304 Not Modified   ETag: "2891734412"
  If-None-Match: "2891734412"                        body: (empty, 0 bytes)
```

How the ETag is computed: a DJB2 hash of the IDs of the newest 20 tweets (`feedHash()`). It's cheap, it's the same input every time, and it changes whenever a new tweet arrives *or* an old one expires. Hashing the IDs is enough, because a tweet's content never changes once posted. You don't need to hash the full body.

INTERVIEW MEIN BOLO

"I moved from polling to an event-driven design. Before, the extension asked the server every couple of seconds, got the whole feed each time, and almost every answer was 'nothing new'. Now the webhook publishes an event to Pusher only when a tweet arrives. Pusher keeps a WebSocket open to each extension and fans the message out. I used Pusher because serverless functions can't hold long-lived connections. For the cases push doesn't cover, I kept one HTTP endpoint with ETag caching. That covers the first load, reconnecting after a dropped connection, and Pusher being unavailable. The client sends the last ETag in `If-None-Match`, and if nothing changed the server replies 304 with an empty body."

WHY KEEP THE CATCH-UP GET AT ALL?

Pusher is fire-and-forget and doesn't replay missed messages. Say the laptop sleeps from 11:00 to 11:20 and three tweets come in. Pusher won't resend them. When the extension reconnects, it makes one catch-up GET and gets those three tweets. `seenIds` makes sure nothing is notified twice.

10. The numbers: 10 GB → ~1 MB, and 6,500+ invocations

SAMJHO

Interviewer zaroor poochega "yeh number kaise nikala?". Formula yaad rakho, aur apne Vercel dashboard (Usage → Fast Data Transfer, Function Invocations) ke real before/after numbers se match karke jaana. Neeche ka calculation ek **worked example** hai; apne actual measured numbers daal ke practise karo.

Bandwidth (outbound data per day)

Formula: bandwidth/day = requests per day × bytes per response

	Before (v1 polling)	After (push + 304)
How data moved	Extension polled every 2 s and got the **whole** feed every time	Pusher sends only new tweets. One catch-up GET on connect. Unchanged GETs = 304.
Requests/day	24 h × 3,600 / 2 s = **43,200**	~1 catch-up per connect (plus 304s)
Size per response	Feed of hundreds of tweets × ~600 B ≈ **~250 KB**	~1 KB per tweet pushed. A 304 has no body.
Per day	43,200 × 250 KB ≈ **10.8 GB**	1,000 tweets × ~1 KB ≈ **~1 MB**

Reduction: (10.8 GB − 1 MB) / 10.8 GB ≈ **99.99%**, which is why the resume says "99%+".

What drove it down, step by step (good to tell as a story):

1. **ETag/304**: an unchanged poll costs headers only, not the feed.
2. **FEED_LIMIT = 50**: the panel shows 5 posts, so the API sends the newest 50, not up to 1,000. A changed feed costs a few KB, not hundreds.
3. **Market-hours gate**: zero requests outside 9:00–15:30 IST.
4. **Pusher**: polling stops entirely while the socket is up. Only the tweets themselves cross the wire.

Serverless invocations eliminated

Formula: invocations/day = active seconds ÷ poll interval (per client)

WORKED EXAMPLE: PLUG IN YOUR REAL INTERVAL

- Market window 9:00–15:30 = 6.5 h = 23,400 s
- Polling every 1.5 s → 23,400 / 1.5 = **15,600** calls/day for one extension
- Polling every 3.5 s → about **6,700** calls/day
- Polling every 10 s (your fallback) → 2,340 per client, × 3 browsers ≈ **7,000**
- After Pusher: ~1 catch-up per connect → **a handful** per day

"6,500+" is a safe, conservative figure for these setups. Use the one that matches what you ran and what your Vercel invocation graph shows.

CAREFUL: THE KEY INSIGHT

A 304 saves **bandwidth** but **not invocations**. The function still runs to compare ETags. That's why ETag alone wasn't enough and you added push. Saying this clearly shows you understand the difference:
"ETag fixed the bytes. Pusher fixed the number of calls."

INTERVIEW MEIN BOLO

"The original client polled every two seconds and downloaded the full feed each time. That's about 43,000 requests a day at a couple of hundred KB each, so over 10 GB a day, almost all of it repeated data. I fixed it in layers. ETags made unchanged responses empty 304s. I capped the response at the newest 50 tweets. I stopped polling outside market hours. Finally I moved to Pusher, so the extension makes one catch-up request when it connects instead of polling. Now only the new tweets themselves cross the network, about a kilobyte each, so around 1 MB a day. One subtle point: 304s save bandwidth, but each one is still a function invocation. Push is what removed the thousands of redundant invocations."

11. Chrome Extension MV3 details

SAMJHO

MV3 (Manifest Version 3) Chrome extensions ka naya standard hai. Sabse badi baat: background script ab ek **service worker** hai, jo Chrome ~30 second idle hone pe **band kar deta hai**. Toh normal `setInterval` pe bharosa nahi kar sakte, aur memory ka data bhi chala jaata hai. Isliye: state `chrome.storage` mein save karte hain, exact time pe uthne ke liye `chrome.alarms` lagate hain, aur ek content script "keep-alive port" kholke rakhta hai taaki session ke beech worker na mare.

MV3 constraint	How you handled it
Service worker can be killed any time	Persist mode, ON-deadline, cached tweets and `seenIds` in `chrome.storage.local`. Reload on start.
Timers don't survive the worker sleeping	`chrome.alarms` for the exact next 8:55 / 9:00 / 15:30 / ON-expiry, plus a 30 s heartbeat
Long WebSocket in a worker	Reconnect after 5 s on close, fall back to polling while disconnected, and a content-script port keeps the worker alive
Permissions are strict	`host_permissions` only for the backend and `*.pusher.com` (https + wss). `storage`, `notifications`, `alarms`, `windows`.
Avoid duplicate poll loops	A "generation counter": a stale tick from an old loop sees an old generation number and exits

INTERVIEW MEIN BOLO

"In MV3 the background script is a service worker that Chrome can stop when it's idle, so I couldn't treat it like a long-running process. All state goes to `chrome.storage`, scheduling uses `chrome.alarms` set to the exact market-open and close times, and the WebSocket reconnects automatically. If Pusher is down, it falls back to slow polling."

12. How to draw the architecture on a whiteboard

SAMJHO

Ek saath poora diagram mat banao. **Story ki tarah, ek-ek box add karo aur bolte jao.** Pehle simple version banao, phir problems batao, phir unka solution add karo. Isse interviewer ko aapki thinking dikhti hai.

1. **Draw the two ends first:** "X accounts" on the left, "Trader's Chrome" on the right. Say: "The goal is to get a post from here to here in under a second."
2. **Add TwitterAPI.io → Webhook (Vercel).** "I don't poll X. The provider calls my webhook."
3. **Add the naive path:** webhook saves to S3, and the extension polls `/api/tweets` every 2 s. Say: "This is v1. It works, but…"
4. **Point out problem 1:** two webhooks at once → lost update. *Write* "If-Match ETag → 412 → retry" on the arrow into S3.

5. **Point out problem 2:** polling wastes bandwidth and calls. First fix: write "ETag / 304" on the poll arrow. "That saves bytes, but every poll still runs a function."
6. **Add Pusher** between the webhook and the extension, in a different colour. "Push on event. Polling becomes one catch-up request on connect."
7. **Add the small boxes:** "normalize + de-dupe" inside the webhook, "seenIds" inside the extension, a clock icon for the market-hours gate.
8. **Finish with the numbers:** "<500 ms delivery, ~10 GB → ~1 MB/day."

The finished board should look like Figure 1. Practise drawing it in under 3 minutes while talking.

13. Interview questions and answers

Design and architecture

Q1. Why a webhook instead of polling the Twitter/X API yourself?
Polling X would mean asking every few seconds for 20 accounts. That's slow, expensive, and runs into rate limits. With a webhook, the provider tells me the moment something is posted, so latency is lower and I do work only when there's data.

Q2. Why S3 and not a real database?
Serverless has no persistent disk. The data is one small daily list, and S3 is cheap, durable and needs no server. It also has native ETags and conditional writes. At bigger scale (per-user feeds, history, queries) I'd move to DynamoDB with one item per tweet.

Q3. Why Pusher instead of your own WebSocket server?
Vercel functions are short-lived and can't hold thousands of open connections. Running my own socket server would mean a separate always-on machine, scaling and reconnect logic. Pusher is managed and handles fan-out. Its free tier allows 200k messages a day, far more than ~1,000 tweets.

Q4. Why not Server-Sent Events (SSE) or long polling?
Both need a server to hold the connection open, and serverless functions have execution time limits and are billed while they're running. Pusher moves that open connection off my backend.

Q5. Walk me through what happens when a tweet is posted.
Provider POSTs to the webhook → secret check → normalize → merge into S3 with If-Match (de-dupe by ID) → publish to Pusher in chunks of 2 → extension receives it over WebSocket → checks seenIds → updates panel and badge → desktop notification.

Q6. What if the extension was offline when a tweet was pushed?
Pusher doesn't replay. On reconnect, the extension makes one catch-up GET to `/api/tweets` with its last ETag. It gets the latest 50 tweets (or a 304 if nothing changed), and `seenIds` prevents duplicate alerts.

Q7. What if Pusher goes down?
The socket's `onclose` handler switches the extension to polling every 10 s (with ETag) and retries Pusher every 5 s. Polling stops again once Pusher reconnects.

Data correctness

Q8. What is idempotency? Is your webhook idempotent?
An operation is idempotent if doing it twice has the same effect as doing it once. Yes: the merge de-dupes by tweet ID, so a retried delivery changes nothing, and I even skip the S3 write when nothing changed.

Q9. Optimistic vs pessimistic locking: when would you pick each?
Optimistic (check the version at write time, retry) when conflicts are rare, as here. Pessimistic (take a lock first) when conflicts are frequent or a retry is very expensive. S3 has no locks, so optimistic fits naturally.

Q10. What if the conditional write keeps failing?
I retry up to 6 times with increasing delay plus jitter. If all six fail, the webhook returns 500, so the provider retries the delivery later. Because the write is idempotent, that retry is safe.

Q11. Is S3 eventually consistent? Could you read stale data?
Since December 2020, S3 gives strong read-after-write consistency for all operations. A GET after a successful PUT returns the new data. My reads also have a deliberate 1-second in-memory cache, so a feed can be up to ~1 s old. That's fine, because real-time delivery goes through Pusher.

Q12. How do you keep tweets in order?
New tweets are put at the front of the list (newest first), and the list is capped at 1,000. The extension merges incoming tweets ahead of its history and de-dupes. For strict ordering you'd sort by `created_at`, but arrival order is good enough for a news feed.

Q13. Why normalize on the server and not in the extension?
A single source of truth. Every consumer (extension, dashboard, future apps) sees one clean shape, and fixing a parsing bug means one deploy instead of an extension update through the Web Store. The extension still normalizes defensively, in case an old payload reaches it.

Q14. How do you delete old data? Is there a cron job?
No cron. The cutoff "08:55 IST today" is computed by a shared function. The webhook drops old rows on write, the API hides them on read, and the extension purges its cache at an exact 08:55 alarm. Computing it everywhere means all layers agree to the second, and there's no job that can silently fail.

Performance and cost

Q15. How did you measure under 500 ms?
Server-side `received_at` timestamp vs extension receive time, sampled across a trading day, quoting p95. The S3 write is the biggest part (~100–200 ms), then the Pusher publish and the WebSocket hop. The X → provider leg is excluded, because it's not under my control.

Q16. Does a 304 response cost anything?
It saves the body bytes, but the serverless function still runs, so it's still an invocation. That's exactly why I moved to push on top of ETags.

Q17. How did you compute the ETag? Why not hash the whole body?
A DJB2 hash of the newest 20 tweet IDs. A tweet's content doesn't change once posted, so the list of IDs fully identifies the feed. Hashing IDs is cheaper than serializing and hashing the whole JSON.

Q18. Why chunk Pusher events into 2 tweets?
Pusher messages are limited to 10 KB. A tweet with media URLs and metadata can be 2–4 KB, so 2 per event stays safely under the limit.

Q19. What does the off-hours behaviour cost?
The provider has no pause button, so it still delivers overnight. The webhook replies with a fast 200 without touching S3, and an optional Cloudflare Worker can answer at the edge so Vercel isn't even invoked. Off-hours GETs without `force` get a CDN-cached empty response (`s-maxage=300`).

Q20. Why does the extension send `?force=1`?
The CDN caches by URL. Without it, a "market closed" response cached at 08:59 could be served back at 09:00, after the market opened. The extension already decides when to poll, so it bypasses that cache. This was a real bug and is a good story.

Security

Q21. How do you stop someone injecting fake "breaking news"?

The webhook needs a secret, compared with `crypto.timingSafeEqual` so attackers can't guess it character by character from response times. With no secret configured, the route refuses all writes, so it fails closed. The feed endpoint checks a poll token or client ID.

Q22. Any security weaknesses you know of?

The secret travels in the URL (the provider's headers aren't documented), so it can end up in logs. I'd move to an HMAC signature header if the provider supports one. The Pusher channel is public. For paid or per-user data I'd use private channels with an auth endpoint. And the client ID check on the feed is weak, so a real per-user token would be better.

Scale and "what next"

Q23. How would this handle 10,000 users?

Reads scale fine: Pusher handles the fan-out, and the catch-up GET can sit behind a CDN keyed on ETag. Writes don't depend on user count, only on tweet volume. Beyond ~5 users I'd move to Pusher's paid tier for connection limits, and add per-user filtering (watchlists) with separate channels.

Q24. And 1,000 accounts / 100k tweets a day?

The single-JSON-object store becomes the bottleneck: every write rewrites the whole file, and conflicts get frequent. I'd put a queue (SQS) between the webhook and storage, store one row per tweet in DynamoDB (conditional put on the tweet ID gives de-dupe for free), and have a worker that publishes to Pusher.

Q25. What would you add for "market sentiment"?

A step after normalize that tags each tweet with a sentiment label (bullish / bearish / neutral) and the stocks mentioned, using an LLM or a finance-tuned classifier. Then filter alerts by the user's watchlist. Run it async, so it doesn't add to the 500 ms path.

Q26. How do you test this?

curl a sample payload to the local webhook (file-store mode, no S3 needed), run the Pusher test script, and use the extension's console commands (`GET_STATUS` , `TEST_NOTIFICATION`). Next I'd add unit tests for `normalizePayload` covering every payload shape, and a test that fires two concurrent writes and checks both tweets are saved.

Behavioural

Q27. Hardest bug you faced?

Pick one and tell it as Situation → Cause → Fix → Result: (a) tweets credited to the wrong account because the matcher searched the tweet body; (b) the CDN serving a stale "closed" response at market open, fixed with `?force=1` ; (c) the lost-update race on S3, fixed with If-Match.

Q28. What trade-off are you least happy with?

Writing to S3 before pushing adds ~100–200 ms to delivery. I chose durability first, so no user sees a tweet the system then loses. If latency mattered more, I'd push and persist in parallel.

Q29. Why did you build this?

Be genuine: for example, you or traders you know were missing market-moving headlines while switching tabs, and existing tools were noisy or slow. Keep it to two sentences.

Q30. If you started again, what would you change?

Start with push from day one instead of polling, store one record per tweet instead of one big file, and add latency and error metrics (p50/p95 delivery time) from the start.

14. Weak spots, trade-offs and what you'd improve

> **SAMJHO**
> Senior interviewers ko yeh sunna achha lagta hai ki aap apne system ki **kamzoriyan khud jaante ho**. Inko pehle hi mention kar do ("one trade-off I made was…"), toh woh trap ki tarah nahi lagega.

Weak spot	Why it's OK for now	Fix at scale
One JSON object holds the whole day	~1,000 tweets, a few hundred KB, rare conflicts	DynamoDB, one item per tweet, or a key per day/hour
Push happens after the S3 write	Durability first; still under 500 ms	Push and persist in parallel
Pusher doesn't replay messages	Catch-up GET on reconnect covers the gap	A replay-capable stream (Ably history, Kafka) if strict delivery is needed
Public Pusher channel	Public news, no user data	Private channels plus an auth endpoint
Webhook secret in URL	The provider doesn't document signature headers	HMAC signature verification
No queue between webhook and store	Low volume; provider retries on 500	SQS + worker to absorb bursts
Latency not continuously tracked	Measured by sampling	Log p50/p95 delivery time per tweet

15. One-page cheat sheet

Term	One-line meaning (say this)
Ingestion pipeline	The steps that bring outside data in: receive → check → clean → de-dupe → store → broadcast.
Webhook	The provider calls my URL when there's new data. Push, not pull.
Normalize	Convert every payload shape into one standard tweet object.
De-duplicate	Keep each tweet once, using a Set of tweet IDs (server and extension).
Idempotent	Doing it twice = doing it once. Safe for retries.
ETag	A version fingerprint of the data.
304 Not Modified	"You already have the latest." Empty body.
If-None-Match	Read header: "send it only if it changed from this ETag."
If-Match	Write header: "save only if it's still this ETag," else 412.
Optimistic concurrency	No locks. Check the version at write time, retry on conflict.
Lost update	Two writers read the same version and the second overwrites the first.
Jitter	A random wait before retrying, so writers don't collide again.
WebSocket	A connection that stays open, so the server can send messages any time.
Pusher	A managed WebSocket service. My server calls `trigger()`, Pusher fans out to clients.
Event-driven	Work happens only when something happens (a new tweet), not on a timer.
Serverless invocation	One run of a function, each counted and billed.
MV3 service worker	The extension background script. Chrome can stop it, so state goes in storage and timing uses alarms.
p95 latency	95% of requests are faster than this number.

Numbers to remember

- **<500 ms** webhook → extension (S3 ~100–200 ms is the biggest part)
- **1,000** max tweets kept per day · **50** per API response · **5** shown in the panel
- **6** conditional-write retries with 40–100 ms jitter · **1 s** read cache
- **2** tweets per Pusher event (10 KB limit) · **10 s** fallback poll · **5 s** reconnect
- **9:00–15:30 IST** active (+5 min grace) · wiped at **08:55** · **5 min** manual ON
- **43,200 × 250 KB ≈ 10.8 GB** before → **~1,000 × 1 KB ≈ 1 MB** after

> **CLOSING LINE**
> "The main lesson was to match the design to how the data actually behaves. Tweets are rare, unpredictable events, so pushing them beats polling for them. And with retries and concurrent webhooks, correctness came from two small ideas: idempotent writes and optimistic concurrency."